作业 HW4* 实验报告

姓名：李欣洋 学号：2453359 日期：2025 年 12 月 1 日

实验报告格式要求按照模板（使用 Markdown 等也请保证报告内包含模板中的要素）
对字体大小、缩进、颜色等不做强制要求（但尽量代码部分和文字内容有一定区分，可参考 vscode 配色）1
实验报告要求在文字简洁的同时将内容表示清楚
报告内不要大段贴代码，尽量控制在 20 页以内

1. 涉及数据结构和相关背景

题目或实验涉及数据结构的相关背景
实验整体使用到的数据结构是图，多使用邻接表和邻接矩阵的形式来存储图。
题目一:
给定一个无向图，用 dfs 和 bfs 找出图的所有连通分量。
所有顶点用 0 到 n-1 表示，搜索时总是从编号最小的顶点出发。使用邻接矩阵存储，或者邻接表（使用邻接表时需要使用尾插法）。
题目二:
给定一个社交网络图，请你对每个节点计算符合“六度空间”理论的结点占结点总数的百分比。
题目三:
N 个村庄，从 1 到 N 编号，现在请你修建一些路使得任何两个村庄都彼此连通。我们称两个村庄 A 和 B 是连通的，当且仅当在 A 和 B 之间存在一条路，或者存在一个村庄 C，使得 A 和 C 之间有一条路，并且 C 和 B 是连通的。
已知在一些村庄之间已经有了一些路，您的工作是再兴建一些路，使得所有的村庄都是连通的，并且新建的路的长度是最小的。
题目四:
给定一个 正 整数 k，同时给你：
一个大小为 n 的二维整数数组 rowConditions，其中 rowConditions[i] = [abovei, belowi] 和
一个大小为 m 的二维整数数组 colConditions，其中 colConditions[i] = [lefti, righti]。
两个数组里的整数都是 1 到 k 之间的数字。
你需要构造一个 k x k 的矩阵，1 到 k 每个数字需要 恰好出现一次。剩余的数字都是 0。
矩阵还需要满足以下条件：
对于所有 0 到 n - 1 之间的下标 i，数字 abovei 所在的 行 必须在数字 belowi 所在行的上面。
对于所有 0 到 m - 1 之间的下标 i，数字 lefti 所在的 列 必须在数字 righti 所在列的左边。
题目五:
某校的计算机系有 n 门必修课程。学生需要修完所有必修课程才能毕业。
每门课程都需要一定的学时去完成。有些课程有前置课程，需要先修完它们才能修这些课程；而其他课程没有。 不同于大多数学校，学生可以在任何时候进行选课，且同时选课的数量没有限制。
现在校方想要知道：
从入学开始，每门课程最早可能完成的时间（单位：学时）；
对每一门课程，若将该课程的学时增加 1，是否会延长入学到毕业的最短时间。
题目六:

假设无向图 G 上有 N 个点和 M 条边，点编号为 1 到 N，第 i 条边长度为 w_i，其中 H 个点上有可以食用的牧草。
另外有 R 匹小马，第 j 匹小马位于点 start_j，需要先前往任意一个有牧草的点进食牧草，然后前往点 end_j，
请你计算每一匹小马需要走过的最短距离。

2. 实验内容

4.1 图的遍历

4.1.1 问题描述

给定一个无向图，用 dfs 和 bfs 找出图的所有连通分量。
所有顶点用 0 到 n-1 表示，搜索时总是从编号最小的顶点出发。使用邻接矩阵存储，或者邻接表（使用邻接表时需要使用尾插法）。

4.1.2 基本要求

简短地描述题目要求
输入:
第 1 行输入 2 个整数 n m，分别表示顶点数和边数，空格分割
后面 m 行，每行输入边的两个顶点编号，空格分割
输出:
第 1 行输出 dfs 的结果
第 2 行输出 bfs 的结果
连通子集输出格式为{v11 v12 ...}{v21 v22 ..}... 连通子集内元素之间用空格分割，子集之间无空格,'{'和子集内第一个数字之间、'}'和子集内最后一个元素之间、子集之间均无空格

4.1.3 数据结构设计

```
vector<bool> visited(n,false);//辅助数组 用于记录该顶点是否被访问过
vector<vector<int> > paragraph(n);//使用邻接表存储图结构
```

4.1.4 功能说明（函数、类）

```
//BFS
void BFS(int start,vector<bool>&visited,vector<vector<int> >&paragraph)
{
    vector<int>path;//记录 BFS 生成的路径
    visited[start]=true;//将当前访问结点标记为已访问
    queue<int> q;//使用队列进行 BFS
    q.push(start);
    while(!q.empty())
    {
        int cur=q.front();//每一次取出队列头的元素
        path.push_back(cur);
```

```
        q.pop();
        for(int val : paragraph[cur])//遍历队列头元素的邻居
        {
            if(!visited[val])//如果邻居未被访问过 则入队
            {
                visited[val]=true;
                q.push(val);
            }
        }
    }

    cout<<"{";
    for(int i=0;i<path.size();i++)
    {
        if(i!=path.size()-1)
          cout<<path[i]<<" ";
        else
          cout<<path[i];
    }
    cout<<"}";
}

//DFS
void DFS(int i,vector<bool>&visited,vector<vector<int> >&paragraph)
{
    if(visited[i]) //DFS 的出口，如果当前点已被访问则退出
      return;
    visited[i]=true;//将当前结点标记为已访问
    temppath.push_back(i);
    for(int val : paragraph[i])//访问邻居
    {
        if(!visited[val])//如果邻居未被访问则对邻居做 DFS
        {
            DFS(val,visited,paragraph);
        }
    }
}
```

4.1.5 调试分析（遇到的问题和解决方法）

此题是一个关于图的遍历的简单问题，因此在实际写代码的过程中并没有遇到错误问题。

4.1.6 总结和体会

1. 题目难点

理解连通分量的概念并正确实现 DFS 和 BFS 两种遍历算法，同时需要掌握邻接表的构建方式，

以及在合适的时机重置 visited 数组以遍历所有连通分量。

2. 边界条件判断

需要处理孤立顶点（度为 0 的单独成分量）、空图情况（m=0）、以及输出格式中空集合的判断。

3. 时间复杂度

总体复杂度为 O(n + m)，其中 DFS 和 BFS 各遍历一次图，每个顶点访问一次 O(n)，每条边最多访问两次 O(m)。

4. 易错点

DFS 和 BFS 之间必须用 fill 重置 visited 数组；无向图构建邻接表时容易忘记双向添加边；BFS 中应在入队时标记 visited 而非出队时，否则会重复入队；输出格式中最后一个元素后不加空格需要特判。

5. 此题的收获

深入理解了 DFS（递归/栈实现）和 BFS（队列实现）的区别与应用场景，掌握了用遍历+标记解决图连通性问题的通用方法，熟练运用了 C++ STL 容器，提升了代码模块化和调试能力。

4.2 小世界现象

4.2.1 问题描述

给定一个社交网络图，请你对每个节点计算符合“六度空间”理论的结点占结点总数的百分比。

4.2.2 基本要求

输入:

第 1 行给出两个正整数，分别表示社交网络图的结点数 N（1<N≤2000，表示人数）、边数 M（≤33×N，表示社交关系数）。

随后的 M 行对应 M 条边，每行给出一对正整数，分别是该条边直接连通的两个结点的编号（节点从 1 到 N 编号）。

输出:

对每个结点输出与该结点距离不超过 6 的结点数占结点总数的百分比，精确到小数点后 2 位。每个结节点输出一行，格式为“结点编号:（空格）百分比%”。

4.2.3 数据结构设计

```
    vector< vector<int> > graph(n+1);
    vector<bool> visited(n+1,false);
```

4.2.4 功能说明（函数、类）

```
int BFS(int start,vector<bool>&visited,vector<vector<int> >&graph)
{
    int PathCount=1;//用于记录符合六度空间理论的点的总数
    visited[start]=true;
    queue<pair<int,int> >q;//这里的 pair 对存储的内容分别是 结点，距离 start 的距离
    q.push(std::make_pair(start,0));
    while(!q.empty())
    {
```

```
        auto temp=q.front();
        int cur=temp.first;
        int depth=temp.second;
        q.pop();
        if(depth==6)//如果当前结点距离 start 的距离已经为 6 则跳过 不执行后续代码
          continue;
        for(int val: graph[cur])//遍历当前结点的邻居
        {
            if(!visited[val])
            {
               visited[val]=true;
               PathCount++;//总数加 1
               q.push(std::make_pair(val,depth+1));//入队 且距离 start 的距离加 1
            }
        }
    }
    return PathCount;//返回距离 start 距离小于等于 6 的点的总数
}
```

4.2.5 调试分析（遇到的问题和解决方法）

遇到的问题在于，此题不难看出需要使用 BFS 来对图进行遍历，因为六度空间理论可以理解为是距离 start 点最短距离小于等于 6 的集合，而 BFS 天生适用于无权图的最短路径问题。但是难点在于怎么确定当前访问的结点距离 start 的距离大于等于 6？所以考虑创建 pair 对，其 first 存储的图的顶点，second 存储的是当前结点距离 start 的距离，然后每次入队都是创建一个 pair 队来入队即可解决问题。

4.2.6 总结和体会

1. 题目难点

理解"六度空间"理论（即 6 步以内可达的节点），需要在 BFS 中同时记录节点和距离信息，使用 pair 存储当前节点及其深度，并在深度达到 6 时停止继续扩展该节点。

2. 边界条件判断

需要处理深度为 6 时的剪枝操作（continue 而非 return），起始节点自身算作 PathCount 初始值为 1，节点编号从 1 到 n 而非 0 到 n-1 需要注意数组大小为 n+1，以及百分比计算时的类型转换。

3. 时间复杂度

总体复杂度为 O(n × (n + m))，对每个节点作为起点执行一次 BFS 为 O(n)次，每次 BFS 最坏情况遍历所有节点和边为 O(n + m)，但实际由于深度限制为 6 会提前剪枝。

4. 易错点

每次 BFS 前必须 reset visited 数组否则会影响后续节点的统计；pair 的使用容易混淆 first 和 second；深度判断应该用 continue 跳过而不是 return 结束函数；百分比计算需要强制类型转换为 float 否则整数除法结果错误；输出格式中%%表示输出百分号需要转义。

5. 此题的收获

学会了在 BFS 中携带额外信息（深度）的方法，掌握了使用 pair 或自定义结构体扩展 BFS 功

能，理解了层次遍历中的剪枝优化技巧，熟练运用了 printf 格式化输出控制小数位数，加深了对社交网络距离度量问题的理解。

4.3 村村通

4.3.1 问题描述

N 个村庄，从 1 到 N 编号，现在请你修建一些路使得任何两个村庄都彼此连通。我们称两个村庄 A 和 B 是连通的，当且仅当在 A 和 B 之间存在一条路，或者存在一个村庄 C，使得 A 和 C 之间有一条路，并且 C 和 B 是连通的。
已知在一些村庄之间已经有了一些路，您的工作是再兴建一些路，使得所有的村庄都是连通的，并且新建的路的长度是最小的。

4.3.2 基本要求

输入:
第一行包含一个整数 n（3<=n<=100），表示村庄数目。
接下来 n 行,每行 n 个非负整数，表示村庄 i 和村庄 j 之间的距离。距离值在[1,1000]之间。
接着是一个整数 m，后面给出 m 行，每行包含两个整数 a,b,(1<=a<b),表示在村庄 a 和 b 之间已经修建了路。

输出:
输出一行，仅有一个整数，表示为使所有的村庄连通，要新建公路的长度的最小值。

4.3.3 数据结构设计

```
struct Edge//边结构体
{
    int u;//边的头
    int v;//边的尾
    int wight;//边的权重
    //对 Edge 类型重载了 < 运算符,便于在后续构建最小生成树的时候直接通过边的权值对边进行排序
    bool operator<(const Edge& other) const;
};
```

4.3.4 功能说明（函数、类）

```
//kruskal 算法类
class kruskal
{
public:
    vector<int> parent;//parent 数组，用于记录各结点的祖先结点
    vector<int> rank;//结点的秩数组，用于记录各结点的秩，用于在 union 联合时判断边的方向
```

```
    kruskal(int n);
    int find(int x);//查找某一节点的祖先结点
    void union_(int x,int y);//将 x 和 y 联合为一条边加入到最小生成树中
};

bool Edge::operator <(const Edge& other) const
{
    return wight < other.wight;
}

kruskal::kruskal(int n)
{
    parent.resize(n + 1);
    rank.resize(n + 1, 0);
    for (int i = 1; i <= n; i++)
        parent[i] = i;//每一个结点的初始祖先都是自己
}

int kruskal::find(int x)
{
    return parent[x] == x ? x : parent[x] = find(parent[x]);//递归查找祖先
}


void kruskal::union_(int x, int y)
{
    int rootX = find(x);//找到 x 的祖先
    int rootY = find(y);//找到 y 的祖先

    if (rootX != rootY)//如果 x 的祖先和 y 的祖先不一致，说明 x 和 y 连接成为边此时并不在
当前的树集合中
    {
        if (rank[rootX] < rank[rootY])//如果 x 的秩小于 y 的秩，则 x 的祖先是 y
        {
            parent[rootX] = rootY;
        }
        else
        {
            parent[rootY] = rootX;//反之 y 的祖先是 x
            if (rank[rootX] == rank[rootY])//在两个点的秩相同时，因为在上一行代码选
择了 x 为祖先
            //所以此时祖先结点秩+1 用于区分
                rank[rootX]++;
        }
    }
```

```
}
//从权重最小的边开始构造最小生成树
    for(auto &edge : edges)
    {
        int u=edge.u;
        int v=edge.v;
        int w=edge.wight;
        //当两个结点的祖先不一致时，加入树集合
        if(ks.find(u)!=ks.find(v))
        {
            ks.union_(u,v);
            totalwight+=w;
        }
    }
```

4.3.5 调试分析（遇到的问题和解决方法）

此题遇到的第一个问题就是如何处理图结构的输入。
因为无向图采取邻接矩阵的方式进行输入，那么这个矩阵必然是一个对称矩阵，所以如果在输入时不进行处理，将上三角矩阵和下三角矩阵同时输入的话，会造成边的重复输入。

```
    for(int i=0;i<n;i++)
    {
        for(int j=0;j<n;j++)
        {
            int w;
            cin>>w;
            if(j<=i) continue;
            if(w!=0)
            {
                Edge e;
                e.u=i+1;
                e.v=j+1;
                e.wight=w;
                edges.push_back(e);
            }
        }
    }
```

因此输入的时候采取如上的方式进行，在 j<=i 的时候，将读取到的元素丢弃然后跳过，这样就可以避免重复加入相同的边导致后续的算法实现发生错误。

4.3.6 总结和体会

1. 题目难点

理解并实现 Kruskal 最小生成树算法，需要掌握并查集（Union-Find）的路径压缩和按秩合并优化，理解如何将已有道路（权重设为 0）融入到算法中，以及处理邻接矩阵的上三角输入。

2. 边界条件判断
需要处理邻接矩阵中 j<=i 的情况避免重复添加边，权重为 0 表示不连通需要跳过（但已有道路的权重也设为 0 需区分），已有道路需要在 edges 中找到对应边并将权重置 0，节点编号从 1 到 n 需要注意数组索引转换。
3. 时间复杂度
总体复杂度为 O(E log E + E × α(n))，其中边排序为 O(E log E)，E 为边数最多为 n×(n-1)/2，遍历所有边并执行 find 操作为 O(E × α(n))，α(n)为阿克曼函数的反函数接近常数。
4. 易错点
find 函数的路径压缩写法容易出错应使用 parent[x] = find(parent[x])而非直接返回；union 时需要先判断 rootX != rootY 避免形成环；读取已有道路后需要遍历 edges 查找对应边并修改权重，正反两个方向都要判断；邻接矩阵输入时容易遗漏 j<=i 的跳过导致边重复；初始化 edges 的大小为 n×(n-1)/2 但后续 push_back 会追加元素。
5. 此题的收获
深入理解了 Kruskal 算法的贪心思想（按权重从小到大选边），掌握了并查集的两大优化技巧（路径压缩和按秩合并）及其时间复杂度分析，学会了最小生成树问题的建模方法，熟练运用了结构体运算符重载简化排序操作，理解了如何将实际约束（已有道路）转化为算法输入。

4.4 给定条件下构造构造矩阵

4.4.1 问题描述

给定一个 正 整数 k，同时给你：
一个大小为 n 的二维整数数组 rowConditions，其中 rowConditions[i] = [abovei, belowi] 和
一个大小为 m 的二维整数数组 colConditions，其中 colConditions[i] = [lefti, righti] 。
两个数组里的整数都是 1 到 k 之间的数字。
你需要构造一个 k x k 的矩阵，1 到 k 每个数字需要 恰好出现一次 。剩余的数字都是 0 。
矩阵还需要满足以下条件：
对于所有 0 到 n - 1 之间的下标 i，数字 abovei 所在的 行 必须在数字 belowi 所在行的上面。
对于所有 0 到 m - 1 之间的下标 i，数字 lefti 所在的 列 必须在数字 righti 所在列的左边。

4.4.2 基本要求

输入：
第一行包含 3 个整数 k、n 和 m
接下来 n 行，每行两个整数 abovei、belowi，描述 rowConditions 数组
接下来 m 行，每行两个整数 lefti、righti，描述 colConditions 数组
输出：
如果可以构造矩阵，打印矩阵；否则输出-1
矩阵中每行元素使用空格分隔

4.4.3 数据结构设计

```
    vector<pair<int,int> > rowConditions(n);//存储题目输入的行约束条件
    vector<pair<int,int> > colConditions(m);//存储题目输入的列约束条件

class Solution
```

```
{
    public:
    int node;
    vector<int>answer;//存储拓扑排序排序后的序列，方便后续输出矩阵的时候使用
    vector<int>indegree;//记录每一个点的入度 便于后续进行拓扑排序
    vector<vector<int> >graph;//记录图结构
    Solution(int k);
    void buildGraph(vector<pair<int,int> >&condtions);//通过 rowCondition 和
colCondition 生成的图结构
    int getanswersize();
    bool topoSort();//拓扑排序函数
};
```

4.4.4 功能说明（函数、类）

```
//对图结构进行拓扑排序
bool Solution::topoSort()
{
    queue<int>q;

    for (int i = 1; i <= node; i++)
    {
        //遇到入度为 0 的结点 可以直接入队
        if (indegree[i] == 0)
            q.push(i);
    }

    while(!q.empty())
    {
        //每一次取出队头元素
        int current=q.front();
        q.pop();
        //放入 answer 数组中，方便后续输出矩阵
        answer.push_back(current);
        //遍历邻居，每次让邻居的入度减 1
        for(int next : graph[current])
        {
            indegree[next]--;
            //如果这一次减 1 之后邻居的入度为 0 了，则邻居入队
            if(indegree[next]==0)
              q.push(next);
        }
    }
    //返回 answer.size() == node 的 bool 值，若值为 true，说明拓扑排序成功
```

```
    //若值为 false，说明图中有环，拓扑排序失败
    return answer.size() == node;
}
//打印矩阵
        vector<vector<int> >matrix(k,vector<int>(k,0));
        vector<int>rowpos(k+1);
        vector<int>colpos(k+1);
        for(int i=0;i<k;i++)
        {
            //记录位置
            rowpos[rowgraph.answer[i]]=i;
            colpos[colgraph.answer[i]]=i;
        }
        for(int i=1;i<=k;i++)
        {
            matrix[rowpos[i]][colpos[i]]=i;
        }
```

4.4.5 调试分析（遇到的问题和解决方法）

此题的难点在于识破题目要求的底层逻辑，对于行上的元素来说，由上到下，呈现出一种排序的关系，在上方的元素总是在在下方的元素之前；而对于列上的元素来说也是一样的。

这样就启发我们，对于行和列来说，各自都是一个拓扑排序序列。此时不难想到，将 rowCondition 和 colCondition 两个数组存储到一个图中对图进行拓扑排序。但是我们后续要如何将这两个拓扑排序序列结合起来进行矩阵的打印？

我们只需要将某个数字在行拓扑排序中的位置作为他的行数，在列拓扑排序中的位置作为他的列数，然后做矩阵的输出即可。

4.4.6 总结和体会

1. 题目难点

理解拓扑排序的原理及其在矩阵构造中的应用，需要将行列约束分别建图并独立进行拓扑排序，关键在于理解如何将拓扑排序结果（线性序列）映射到矩阵的行列位置，以及判断图中是否存在环导致无解。

2. 边界条件判断

需要判断拓扑排序是否成功（answer.size() == node），若任一方向存在环则返回-1，节点编号从 1 到 k 需要注意数组大小为 k+1，rowpos 和 colpos 数组用于记录每个数字在矩阵中的位置索引从 0 开始。

3. 时间复杂度

总体复杂度为 O(k + n + m)，其中 buildGraph 构建图为 O(n)和 O(m)，拓扑排序遍历所有节点和边为 O(k + n)和 O(k + m)，构造矩阵为 $O(k^2)$，但实际瓶颈在矩阵输出的 $O(k^2)$。

4. 易错点

reserve 和 resize 容易混淆应使用 resize 预分配空间并初始化，拓扑排序中入度减为 0 才入队而非直接入队所有邻居，answer 数组存储的是排序后的序列需要再次映射到 rowpos/colpos 才能确定矩阵位置，判断无解时两个方向都要检查不能遗漏，矩阵输出时注意空格格式每行末尾有空格。

5. 此题的收获

深入理解了拓扑排序解决依赖关系问题的思想（入度为 0 的节点优先处理），掌握了 Kahn 算法（BFS 实现拓扑排序）的完整流程，学会了将二维约束问题分解为两个独立的一维拓扑排序，熟练运用了 pair 存储约束条件简化数据结构，理解了通过位置映射数组将线性序列转换为二维矩阵的技巧。

4.5 必修课

4.5.1 问题描述

某校的计算机系有 n 门必修课程。学生需要修完所有必修课程才能毕业。
每门课程都需要一定的学时去完成。有些课程有前置课程，需要先修完它们才能修这些课程；而其他课程没有。 不同于大多数学校，学生可以在任何时候进行选课，且同时选课的数量没有限制。
现在校方想要知道：
从入学开始，每门课程最早可能完成的时间（单位：学时）；
对每一门课程，若将该课程的学时增加 1，是否会延长入学到毕业的最短时间。

4.5.2 基本要求

输入：
第一行，一个正整数 n，代表课程的数量。
接下来 n 行，每行若干个整数：

- 第一个整数为 ti，表示修完该课程所需的学时。
- 第二个整数为 ci，表示该课程的前置课程数量。
- 接下来 ci 个互不相同的整数，表示该课程的前置课程的编号。

该校保证，每名入学的学生，一定能够在有限的时间内毕业。

输出：
输出共 n 行，第 i 行包含两个整数：

- 第一个整数表示编号为 i 的课程最早可能完成的时间。
- 第二个整数表示，如果将该课程的学时增加 1，入学到毕业的最短时间是否会增加。如果会增加则输出 1，否则输出 0。

每行的两个整数以一个空格隔开。

4.5.3 数据结构设计

```
struct Edge
{
    int u;//前置课程
```

```
    int v;//后置课程
    int wight;//学时
};
vector<Edge>graph(n*(n-1)/2);//存储图结构
vector<int>indegree(n+1,0);//存储入度
vector<int>outdegree(n+1,0);//存储出度
```

4.5.4 功能说明（函数、类）

```
//通过 dp 计算活动的最早发生时间
void dp_ve(vector<int>&ve,vector<int>&topo_order,vector<Edge>&graph)
{
    for(int u : topo_order)
    {
        for(auto&edge : graph)
        {
            if(edge.u==u)
            {
                //由 u 指向 v  v 的最早发生时间是 当前已有的 v 的最早发生时间 和 u 的最早发生时间 + 边的权重 的最大值
                ve[edge.v]=max(ve[edge.v],ve[u]+edge.wight);
            }
        }
    }
}

//dp 计算活动的最迟发生时间
void dp_vl(vector<int>&vl,vector<int>&retopo_order,vector<Edge>&graph,int max_time)
{
    //先做一步初始化，这里的 max_time 是 ve 中的最大值，因为汇点的最早发生时间和最迟发生时间是一样的
    for(int i=0;i<vl.size();i++)
    {
        vl[i]=max_time;
    }
    for(int v : retopo_order)
    {
        for (auto& edge : graph)
        {
            if (edge.v == v)
            {
```

```
                //由 u 指向 v 的边 u 的最迟发生时间等于 当前已有的 u 的最迟发生时间 和 v 的最迟发生时间减去边的权重 的最小值
                vl[edge.u] = min(vl[edge.u], vl[edge.v] - edge.wight);
            }
        }
    }
}
    //计算事件的最早发生时间和最迟发生时间 活动盈余等于 l-e l-e 等于 0 的点在关键路径上
    for(int i=0;i<n+1;i++)
    {
        e[graph[i].v]=ve[graph[i].u];
        l[graph[i].v]=vl[graph[i].v]-graph[i].wight;
    }
```

4.5.5 调试分析（遇到的问题和解决方法）

此题难点在 理解 ve 和 vl 的实际意义。最早发生时间和最迟发生时间，这里的最早在一开始的时候难以理解，ve 是事件最早发生时间（从源点出发能最早到达的时间），vl 是事件最迟发生时间（不影响工期的最晚开始时间），容易搞反两者的更新方式（ve 用 max，vl 用 min）。

4.5.6 总结和体会

1. 题目难点
理解关键路径算法在 AOE 网中的应用，需要掌握正向拓扑排序计算最早发生时间 ve 和逆向拓扑排序计算最迟发生时间 vl，关键在于理解 ve[i]==vl[i]时该课程在关键路径上，增加学时会延长毕业时间。
2. 边界条件判断
需要处理没有前置课程的情况。
3. 时间复杂度
总体复杂度为 O(n × E)，其中 E 为边数，正向和逆向拓扑排序各为 O(n + E)。
4. 易错点
vl 初始化必须为 max_time 而非 0 否则 min 操作失效。
5. 此题的收获
深入理解了 AOE 网的关键路径算法原理，掌握了通过正向 DP 求最早时间和逆向 DP 求最迟时间的对偶思想，学会了识别关键活动（ve==vl 的节点）来判断哪些课程影响毕业时间，熟练运用了虚拟源点和汇点简化无前驱和无后继节点的处理，理解了拓扑排序在项目调度和依赖管理中的实际应用价值。

4.6 小马吃草

4.6.1 问题描述

假设无向图 G 上有 N 个点和 M 条边，点编号为 1 到 N，第 i 条边长度为 w_i，其中 H 个点上有可以食用的牧草。
另外有 R 匹小马，第 j 匹小马位于点 start_j，需要先前往任意一个有牧草的点进食牧草，然后前往点 end_j，

请你计算每一匹小马需要走过的最短距离。

4.6.2 基本要求

输入:
第一行两个整数 N、M，分别表示点和边的数量
接下来 M 行，第 i 行包含三个整数 x_i,y_i,w_i，表示从点 x_i 到点 y_i 有一条长度为 w_i 的边，保证 x_i≠y_i
接下来一行有两个整数 H 和 R，分别表示有牧草的点数量和小马的数量
接下来一行包含 H 个整数，为 H 个有牧草的点编号
接下来 R 行，第 j 行包含两个整数 start_j 和 end_j，表示第 j 匹小马起始位置和终点位置
题目保证两个点之间一定是连通的，并且至少有一个点上有牧草

输出:
输出共 R 行，表示 R 匹小马需要走过的最短距离

4.6.3 数据结构设计

```
// 邻接表结构
struct Edge
{
    int v;      // 终点
    int weight; // 边权
};
```

4.6.4 功能说明（函数、类）

```
//Dijkstra 算法
void Dijkstra(int start_node, int N, const vector<vector<Edge>>& adj,
vector<int>& path_length)
{
    // 初始化距离数组
    fill(path_length.begin(), path_length.end(), I_MAX);

    // pair<distance, node>
    priority_queue<pair<int, int>, vector<pair<int, int>>, greater<pair<int,
int>>> pq;

    //初始点到自己的距离是 0
    path_length[start_node] = 0;

    pq.push(std::make_pair(0, start_node));

    while (!pq.empty())
     {
        auto temp = pq.top();
```

```
        pq.pop();

        int d = temp.first; // 距离
        int u = temp.second; // 节点

        //如果当前取出的距离 d 比已知最短距离大，则忽略
        if (d > path_length[u])
        {
            continue;
        }

        for (const auto& edge : adj[u])
        {
            int v = edge.v;
            int w = edge.weight;

            //如果起始点到当前结点 u 的距离加上权重 小于 v 点的路径长度 则更新 v 点的路径长度 然后入队
            if ((long long)path_length[u] + w < path_length[v]) //用 long long 是防止溢出
            {
                path_length[v] = path_length[u] + w;
                pq.push(std::make_pair(path_length[v], v));
            }
        }
    }
}
```

4.6.5 调试分析（遇到的问题和解决方法）

这道题目在实际写的过程中比较顺利，但是在最后 OJ 平替测试的时候发生了严重超时。

后来经过思考，原因在于，我一开始的思路是，遍历图中的每一个顶点，对于每一个顶点都记录其到其他各个顶点的最短距离，然后再根据小马的起始位置和初始位置来进行计算，选择最短的。

但是这样的遍历在图的结点数较大的时候会发生严重的超时，因为我忽略了无向图的一个重要特性，（a，b）和（b，a）的长度是一样的。因此我们的核心思想应该是"从牧草点出发"而非"从小马出发"，如果对每匹小马单独跑 Dijkstra 会超时，这个视角转换是关键。只对牧草点做 Dijkstra 算法，这样能节约很多的时间。

4.6.6 总结和体会

1. 题目难点

理解多源最短路径问题的优化思路，需要将问题转化为从 H 个牧草点分别运行 Dijkstra 算法，然后对每匹小马枚举所有牧草点找最小值（start→grass_i→end），关键在于避免对每匹小马单独求最短路造成的重复计算。

2. 边界条件判断

需要处理距离为无穷大 I_MAX 的情况避免相加溢出，使用 long long 进行加法运算防止整数溢出，判断路径存在性时需检查 d_start_to_grass 和 d_grass_to_end 都不等于 I_MAX，优先队列中可能有过时的距离需要用 d > path_length[u]剪枝。

3. 时间复杂度

总体复杂度为 O(H × (M log N) + R × H)，其中对 H 个牧草点各运行一次 Dijkstra 为 O(H × (M log N))，M 为边数使用优先队列优化，对 R 匹小马各枚举 H 个牧草点为 O(R × H)。

4. 易错点

Dijkstra 中 distance 和 node 在 pair 中的顺序容易混淆应为<distance, node>便于优先队列按距离排序，无向图添加边时容易遗漏反向边导致图不连通，整数相加溢出需要强制转换为 long long 再比较，dist_from_grass 数组的索引容易混淆应为 dist_from_grass[j][node]表示从牧草点 j 到 node 的距离，优先队列未设置 greater 比较器会变成最大堆而非最小堆。

5. 此题的收获

深入理解了 Dijkstra 算法求单源最短路径的原理及优先队列优化，掌握了多源最短路径问题的预处理思想（提前计算 H 个源点到所有点的距离避免重复），学会了通过转换问题视角优化算法（从小马视角转为牧草点视角），熟练运用了邻接表和优先队列的 STL 实现，理解了在实际问题中如何权衡时间复杂度选择合适的算法策略。

3.实验总结

1. 思维的改进

问题转化能力：通过六道题目学会了将复杂问题转化为图论模型的思维方式，例如将村庄修路问题转化为最小生成树，将课程依赖关系转化为拓扑排序，将小马觅食问题转化为多源最短路径。同时掌握了逆向思维和对偶思维，比如在关键路径问题中正向计算最早时间和逆向计算最迟时间，在最短路径问题中从"小马视角"转换为"牧草点视角"实现优化。这种灵活转换问题视角的能力是解决图论问题的关键。

2. 算法的学习

图遍历算法体系：系统掌握了 DFS 和 BFS 两种基础遍历算法及其应用场景，DFS 适合连通性判断和路径搜索，BFS 适合层次遍历和最短步数问题。学会了在 BFS 中携带额外信息（如深度、距离）来扩展功能。**高级图算法精通**：深入理解了 Kruskal 最小生成树算法的贪心思想和并查集优化（路径压缩+按秩合并），掌握了拓扑排序解决依赖关系问题的 Kahn 算法，理解了关键路径算法在 AOE 网中的应用，熟练运用了 Dijkstra 单源最短路径算法及优先队列优化。这些算法的时间复杂度分析和适用场景判断能力得到了提升。

3. 收获

数据结构与算法的协同运用：学会了根据问题特点选择合适的数据结构，如用邻接表存储稀疏图提高遍历效率，用优先队列优化 Dijkstra 算法，用 pair 存储复合信息简化代码，用并查集高效处理动态连通性。